

CyberAudit

Training Modules Page

Cybersecurity Awareness : 5 Modules · Progress Bar · Start / Continue / Review

Project / Projet	CyberAudit Platform
File	src/pages/TrainingPage.jsx
Route	/training (protected, authenticated clients)
Module statuses	to_start → in_progress → completed
API functions	getTrainingModules(), updateModuleStatus(id, status)
Progress bar	completedCount / total * 100, rounded. CSS transition-all
Button labels	to_start → 'Start' in_progress → 'Continue' completed → 'Review'
Grid layout	1 col mobile → sm:2 cols → lg:3 cols (Tailwind responsive)
Date	June 2026 / Juin 2026

1) Page Overview

1) Vue d'ensemble de la page

TrainingPage is a client-facing awareness training hub. It displays all cybersecurity training modules fetched from the API, a global progress bar showing completion across all modules, and individual module cards each with a status badge and a context-sensitive action button. Completing all modules reaches 100%.

TrainingPage est un hub de formation de sensibilisation à destination des clients. Elle affiche tous les modules de formation à la cybersécurité récupérés depuis l'API, une barre de progression globale montrant l'avancement sur tous les modules, et des cartes de modules individuels avec un badge de statut et un bouton d'action contextuel. Compléter tous les modules atteint 100%.

Page structure / Structure de la page

Page header	Title 'Awareness & Training' + italic subtitle. Static text. Titre 'Awareness & Training' + sous-titre italique. Texte statique.
Progress card	White card with 'My progress' label, completedCount/total counter, and animated progress bar. Carte blanche avec le label 'My progress', compteur completedCount/total, et barre de progression animée.
Module grid	Responsive card grid: 1/2/3 columns. One card per module from the modules array. Grille de cartes responsive : 1/2/3 colonnes. Une carte par module du tableau modules.
Notice banner	Amber banner : 'The detailed content of the modules will be added later.' Static placeholder. Bandeau ambre : 'The detailed content of the modules will be added later.' Espace reservatoire statique.

State variables / Variables d'état

modules	Array<TrainingModule> fetched from getTrainingModules(). Updated in-place after each status advance via updateModuleStatus(). Tableau récupère depuis getTrainingModules(). Mis à jour en place après chaque avancement de statut via updateModuleStatus().
loading	Boolean : true while initial data fetch is in flight. Shows spinner, then replaced by the page. Boolean : true pendant le chargement initial. Affiche le spinner, puis remplace par la page.

2) Data Fetch & Module Shape

2) Récupération des données et structure d'un module

Modules are fetched once on component mount via `useEffect`. The API returns an array of training module objects. If the response is not an array (e.g. paginated), it defaults to empty array to prevent runtime errors. No re-fetch occurs on status change, `updateModuleStatus()` returns the full updated list directly.

Les modules sont récupérés une fois au montage du composant via `useEffect`. L'API retourne un tableau d'objets modules de formation. Si la réponse n'est pas un tableau (ex. `pagine`), elle se rabat sur un tableau vide pour éviter les erreurs runtime. Aucun rechargement n'a lieu lors d'un changement de statut, `updateModuleStatus()` retourne directement la liste complète mise à jour.

```
// TrainingPage.jsx - data fetch on mount
useEffect(() => {
  async function load() {
    const data = await getTrainingModules();
    // Guard: API might return { results: [] } on paginated endpoints
    setModules(Array.isArray(data) ? data : []);
    setLoading(false);
  }
  load();
}, []); // empty deps - one-time fetch on mount
```

Training module object shape / Structure d'un module de formation

id	Numeric primary key from Django. Used as React key and passed to <code>updateModuleStatus()</code>. Clé primaire numérique depuis Django. Utilise comme key React et passe à <code>updateModuleStatus()</code>.
title	Module title displayed as card heading (font-bold text-brand). Titre du module affiche comme en-tête de carte.
description	Short module description. flex-1 in the card, stretches to fill available card height. Courte description du module. flex-1 dans la carte, s'étire pour remplir la hauteur disponible.
status	One of: 'to_start' 'in_progress' 'completed'. Drives badge colour and button label. Un parmi : 'to_start' 'in_progress' 'completed'. Pilote la couleur du badge et le libelle du bouton.

5 awareness modules / 5 modules de sensibilisation

The 5 modules are seeded in the Django database (management command or fixture). Their content is currently placeholder detailed videos and documentation are planned for a future release. The titles and descriptions come from the API; nothing is hardcoded in the React component.

Les 5 modules sont initialisés dans la base Django (commande de gestion ou fixture). Leur contenu est actuellement un espace réservataire des vidéos détaillées et de la documentation sont prévues pour une future version. Les titres et descriptions viennent de l'API ; rien n'est codé en dur dans le composant React.

#	Example title (seeded)	Topic area EN / FR
1	Phishing Awareness	Recognising fraudulent emails, links, and social engineering attacks. Reconnaître les emails frauduleux, liens et attaques d'ingénierie sociale.
2	Password Security	Strong password policies, password managers, multi-factor authentication. Politiques de mots de passe forts, gestionnaires de mots de passe, authentification multi facteur.
3	Safe Browsing	Secure web habits: HTTPS, avoiding malicious sites, browser hygiene. Habitudes web sécurisées : HTTPS, éviter les sites malveillants, hygiène du navigateur.
4	Data Protection	GDPR basics, handling personal data, data classification principles. Base du RGPD, traitement des données personnelles, principes de classification des données.
5	Incident Response	What to do when a security incident occurs: report, contain, document. Que faire en cas d'incident de sécurité : signaler, confiner, documenter.

3) Status System & Transitions

3) Systeme de statut et transitions

Each module has exactly three possible statuses forming a linear one-way progression: `to_start` → `in_progress` → `completed`. Once completed, the status never regresses. `STATUS_LABELS` maps each status to a display label and a Tailwind colour class.

Chaque module a exactement trois statuts possibles formant une progression linéaire à sens unique : `to_start` → `in_progress` → `completed`. Une fois complète, le statut ne régresse jamais. `STATUS_LABELS` mappe chaque statut sur un libelle d'affichage et une classe de couleur Tailwind.

Status value	Label	Tailwind colour	Button + style EN / FR
<code>to_start</code>	To start	<code>text-gray-500</code>	'Start' : <code>bg-brand text-white hover:bg-brand-dark</code> 'Start' : fond brand blanc survol brand-dark
<code>in_progress</code>	In progress	<code>text-amber-500</code>	'Continue' : <code>bg-brand text-white hover:bg-brand-dark</code> 'Continue' : fond brand blanc survol brand-dark
<code>completed</code>	Completed	<code>text-green-600</code>	'Review' : <code>bg-gray-100 text-gray-600 hover:bg-gray-200 (disabled-looking)</code> 'Review' : fond gris clair, texte gris, survol gris (aspect désactive)

handleAdvance() : status transition logic

```
// TrainingPage.jsx - handleAdvance()
async function handleAdvance(module) {
  // Guard: clicking "Review" on a completed module is a no-op
  if (module.status === "completed") return;

  // Linear one-way progression
  const next = module.status === "to_start" ? "in_progress" : "completed";
  // to_start    -> in_progress
  // in_progress -> completed
  // completed  -> (blocked by guard above)

  // updateModuleStatus returns the full refreshed modules array
  const updated = await updateModuleStatus(module.id, next);
  setModules(Array.isArray(updated) ? updated : []);
  // No separate reload needed - server returns full list
}
```

buttonLabel() : context-sensitive label

```
// TrainingPage.jsx - buttonLabel()
function buttonLabel(status) {
  if (status === "completed") return "Review"; // grey style
  if (status === "in_progress") return "Continue"; // brand style
  return "Start"; // brand style (default)
}

// Button style logic in JSX:
// completed -> "bg-gray-100 text-gray-600 hover:bg-gray-200"
// to_start / in_progress -> "bg-brand text-white hover:bg-brand-dark"
```

STATUS_LABELS map / Map STATUS_LABELS

```
// TrainingPage.jsx - STATUS_LABELS constant
const STATUS_LABELS = {
  completed: { label: "Completed", classes: "text-green-600" },
  in_progress: { label: "In progress", classes: "text-amber-500" },
  to_start: { label: "To start", classes: "text-gray-500" },
};

// Usage in JSX:
const statusInfo = STATUS_LABELS[module.status] || STATUS_LABELS.to_start;
// Fallback to to_start avoids crash if API returns an unknown status value
```

4) Progress Bar

4) Barre de progression

The progress bar sits in its own white card at the top of the page. It is computed entirely client-side from the modules array, no dedicated API call. The bar width is driven by an inline style (width: percent%) with a CSS transition-all for smooth animation whenever a module is completed.

La barre de progression est dans sa propre carte blanche en haut de la page. Elle est calculée entièrement cote client depuis le tableau modules, aucun appel API dédié. La largeur de la barre est pilotée par un style inline (`width: percent%`) avec une transition CSS `transition-all` pour une animation fluide à chaque complétion de module.

Progress bar JSX / JSX de la barre de progression

```
// TrainingPage.jsx - progress bar render
<div className="rounded-xl border border-gray-100 bg-white p-5 shadow-sm">
  <div className="flex items-center justify-between">
    <span className="font-bold text-brand">My progress</span>
    <span className="text-sm text-gray-500">
      {completedCount} / {total} modules completed
    </span>
  </div>
  <div className="mt-2 h-3 w-full overflow-hidden rounded-full bg-gray-200">
    <div className="h-full rounded-full bg-brand transition-all"
      style={{ width: `${percent}%` }}
    </div>
  </div>
</div>
```

Progress scenarios / Scenarios de progression

completedCount	total	percent	Display EN / FR
0	5	0%	'0 / 5 modules completed', bar empty '0 / 5 modules completed', barre vide
1	5	20%	'1 / 5 modules completed', bar at 20%. '1 / 5 modules completed', barre a 20%
3	5	60%	'3 / 5 modules completed', bar at 60% '3 / 5 modules completed', barre a 60%
5	5	100%	'5 / 5 modules completed', bar full (brand colour) '5 / 5 modules completed', barre pleine (couleur brand)
0	0	0%	Guard: total > 0 prevents NaN, bar stays empty Garde : total > 0 evite NaN, barre reste vide

5) Module Card UI

5) Interface de la carte de module

Each training module renders as a flex-col card. The description uses flex-1 to push the status badge and button to the bottom of the card regardless of description length, all cards in the same row share the same height via CSS grid stretch.

Chaque module de formation se rend comme une carte flex-col. La description utilise flex-1 pour pousser le badge de statut et le bouton vers le bas de la carte indépendamment de la longueur de la description, toutes les cartes de la même ligne partagent la même hauteur via l'étirement de la grille CSS.

```
// TrainingPage.jsx — module card JSX
{modules.map((module) => {
  const statusInfo = STATUS_LABELS[module.status] || STATUS_LABELS.to_start;
  return (
    <div key={module.id}
      className="flex flex-col rounded-xl border border-gray-100 bg-white p-5
      shadow-sm">

      {/* Card header */}
      <h2 className="font-bold text-brand">{module.title}</h2>

      {/* Description — flex-1 fills remaining space */}
      <p className="mt-1 flex-1 text-sm text-gray-600">{module.description}</p>

      {/* Status badge */}
      <p className="mt-3 text-sm">
        <span className="text-gray-500">Status: </span>
        <span className={`font-semibold ${statusInfo.classes}`}>
          {statusInfo.label}
        </span>
      </p>

      {/* Action button — style depends on status */}
      <button type="button" onClick={() => handleAdvance(module)}
        className={`mt-3 rounded-md py-2 text-sm font-semibold transition ${
          module.status === "completed"
            ? "bg-gray-100 text-gray-600 hover:bg-gray-200"
            : "bg-brand text-white hover:bg-brand-dark"
        }`>
        {buttonLabel(module.status)}
      </button>
    </div>
  );
})}
```

Card layout anatomy / Anatomie de la mise en page de la carte

flex flex-col	Card is a vertical flex container. Children stack top-to-bottom. La carte est un conteneur flex vertical. Les enfants s'empilent de haut en bas.
h2 : title	font-bold text-brand. Module title from API. Fixed height at top of card. font-bold text-brand. Titre du module depuis l'API. Hauteur fixe en haut de la carte.
p : description	flex-1: absorbs all available vertical space. Pushes badge+button to card bottom. flex-1 : absorbe tout l'espace vertical disponible. Pousse badge+bouton vers le bas de la carte.
Status badge	'Status: ' (gray-500) + label (colored via statusInfo.classes). Below description. 'Status: ' (gris-500) + libelle (colore via statusInfo.classes). Sous la description.
Action button	Full-width (no explicit w-full, fills column). mt-3 spacing. Color switches on status. Pleine largeur (remplit la colonne). Espacement mt-3. Couleur change selon le statut.

Responsive grid / Grille responsive

Mobile (< sm)	1 column: grid gap-4 1 colonne : grid gap-4
Tablet (sm: ≥ 640px)	2 columns: sm:grid-cols-2 2 colonnes : sm:grid-cols-2
Desktop (lg: ≥ 1024px)	3 columns: lg:grid-cols-3 3 colonnes : lg:grid-cols-3

6) API Layer : dataService.js

6) Couche API : dataService.js

Two dataService functions handle training data. Both are called from TrainingPage.

getTrainingModules() is used once on mount. updateModuleStatus() is called each time the user clicks a module button and returns the full refreshed list, avoiding a second API call.

Deux fonctions dataService gèrent les données de formation. Les deux sont appelées depuis TrainingPage. getTrainingModules() est utilisée une fois au montage. updateModuleStatus() est appelée à chaque clic sur le bouton d'un module et retourne la liste complète rafraîchie, évitant un second appel API.

getTrainingModules()	GET /api/training-modules/ : returns Array<TrainingModule>. Called once on mount. Returns [] on error or non-array response. GET /api/training-modules/ : retourne Array<TrainingModule>. Appelée une fois au montage. Retourne [] en cas d'erreur ou réponse non-tableau.
updateModuleStatus(id, status)	PATCH /api/training-modules/:id/ updates status field. Returns full refreshed modules array. Allows TrainingPage to replace modules state in one operation. PATCH /api/training-modules/:id/ met à jour le champ status. Retourne le tableau complet des modules rafraîchi. Permet à TrainingPage de remplacer l'état modules en une opération.

update flow / Flux de mise à jour

```
// Full update flow when user clicks a module button

// 1. User clicks "Start" on a to_start module
handleAdvance(module)

// 2. Guard check - completed modules blocked here
// 3. Compute next status
const next = "in_progress"; // to_start -> in_progress

// 4. PATCH to Django REST API
const updated = await updateModuleStatus(module.id, "in_progress");
// PATCH /api/training-modules/3/
// body: { status: "in_progress" }
// response: [{ id: 1, ... }, { id: 2, ... }, { id: 3, status: "in_progress", ... }, ...]

// 5. Replace full modules array in state
setModules(Array.isArray(updated) ? updated : []);

// 6. React re-renders:
// - module card status badge: "To start" -> "In progress" (amber)
// - module button: "Start" -> "Continue" (brand style)
// - progress bar: unchanged (status moved from to_start to in_progress, not completed)
```


7) Summary

7) Resume

Concept	Key detail EN / FR
5 seeded modules	Titles and descriptions from API (Django DB). React hardcodes nothing fully data-driven. Titres et descriptions depuis l'API. React ne code rien en dur entièrement pilote par les données.
3 statuses	to_start → in_progress → completed. Linear, one-way. Completed is terminal. to_start → in_progress → completed. Linéaire, sens unique. Completed est terminal.
STATUS_LABELS	Maps status to { label, classes }. Fallback to to_start on unknown status. Mappe le statut sur { label, classes }. Repli sur to_start si statut inconnu.
buttonLabel()	'Start' 'Continue' 'Review' per status. Completed = grey style, others = brand style. 'Start' 'Continue' 'Review' par statut. Completed = style gris, autres = style brand.
handleAdvance()	Guard on completed. Next = to_start->in_progress or in_progress->completed. updateModuleStatus() returns full list. Garde sur completed. Next = to_start->in_progress ou in_progress->completed. updateModuleStatus() retourne la liste complete.
Progress bar	completedCount/total*100 rounded. Guard against /0. Inline style width. CSS transition-all. completedCount/total*100 arrondi. Garde contre /0. Style inline width. CSS transition-all.
Card layout	flex-col card. Description flex-1 pushes badge+button to bottom. Equal-height via grid stretch. Carte flex-col. Description flex-1 pousse badge+bouton vers le bas. Hauteur egale via l'etirement de la grille.
Responsive grid	1 col → sm:2 cols → lg:3 cols. gap-4 spacing. 1 col → sm:2 cols → lg:3 cols. Espacement gap-4.
No re-fetch	updateModuleStatus() returns full updated list, no second GET needed after status change. updateModuleStatus() retourne la liste complète mise à jour, pas de second GET apres un changement de statut.
Placeholder notice	Amber banner: module content (videos, docs) is planned but not yet implemented. Bandeau ambre : le contenu des modules (videos, docs) est prévu mais pas encore implémente.

Full flow: mount → getTrainingModules() → render 5 cards + progress bar at 0%. User clicks 'Start' → handleAdvance() → PATCH → updateModuleStatus() returns full list → setModules() → card shows 'In progress' (amber) + 'Continue' button. User clicks 'Continue' → 'Completed' (green) + 'Review' button (grey) + progress bar animates to 20%. Repeat x5 → 100%.

Flux complet : montage → getTrainingModules() → rendu 5 cartes + barre à 0%. L'utilisateur clique 'Start' → handleAdvance() → PATCH → updateModuleStatus() retourne la liste → setModules() → carte affiche 'In progress' (ambre) + bouton 'Continue'. L'utilisateur clique 'Continue' → 'Completed' (vert) + bouton 'Review' (gris) + barre animée a 20%. Répéter x5 → 100%.